

CIS 5357

Computing for Data Analytics

Week 4

8 FEBRUARY 2024

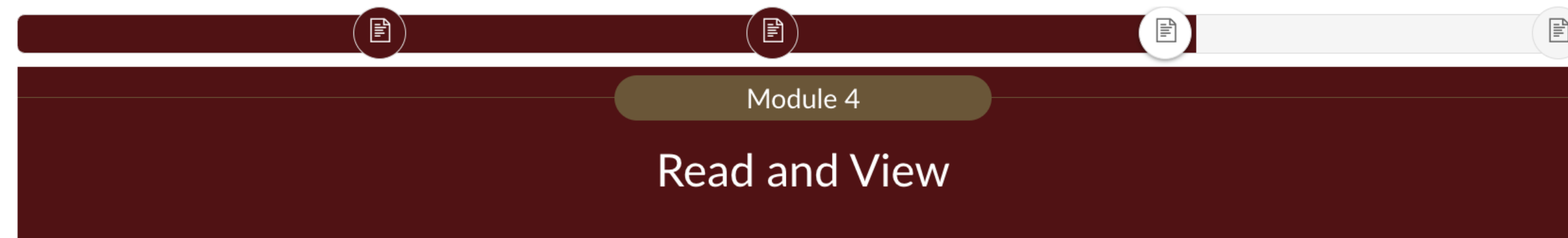
Last week

- selection and decision statements in Python
- if elif else logic
- exception handling
- logic quiz

This week

This week we will explore Python **lists** and learn about how to apply **looping logic** in Python

Download code to follow along



Read

- Textbook chapter 3 pages 88-106
- [Python lists](#) ↗
- [Python for statements](#) ↗ (sections 4.2 through 4.5 only)

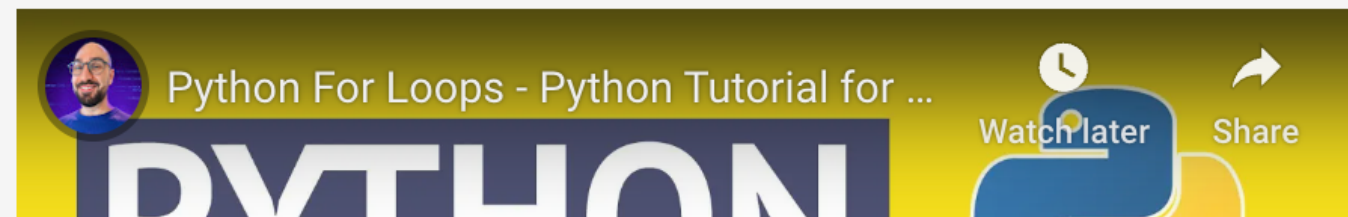
For reference:

- [Practice code from lecture](#) ↓
- Lecture slides (coming soon!)



View

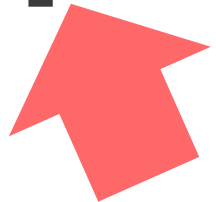
[Python For Loops](#) ↗



Lists

Python lists are a set of comma-separated values (items) between square brackets

```
my_classes = ["Computing for Data Analytics", "Database  
Management Systems", "Statistical Methods for Business"]
```



Lists start and
end with square
brackets

Lists

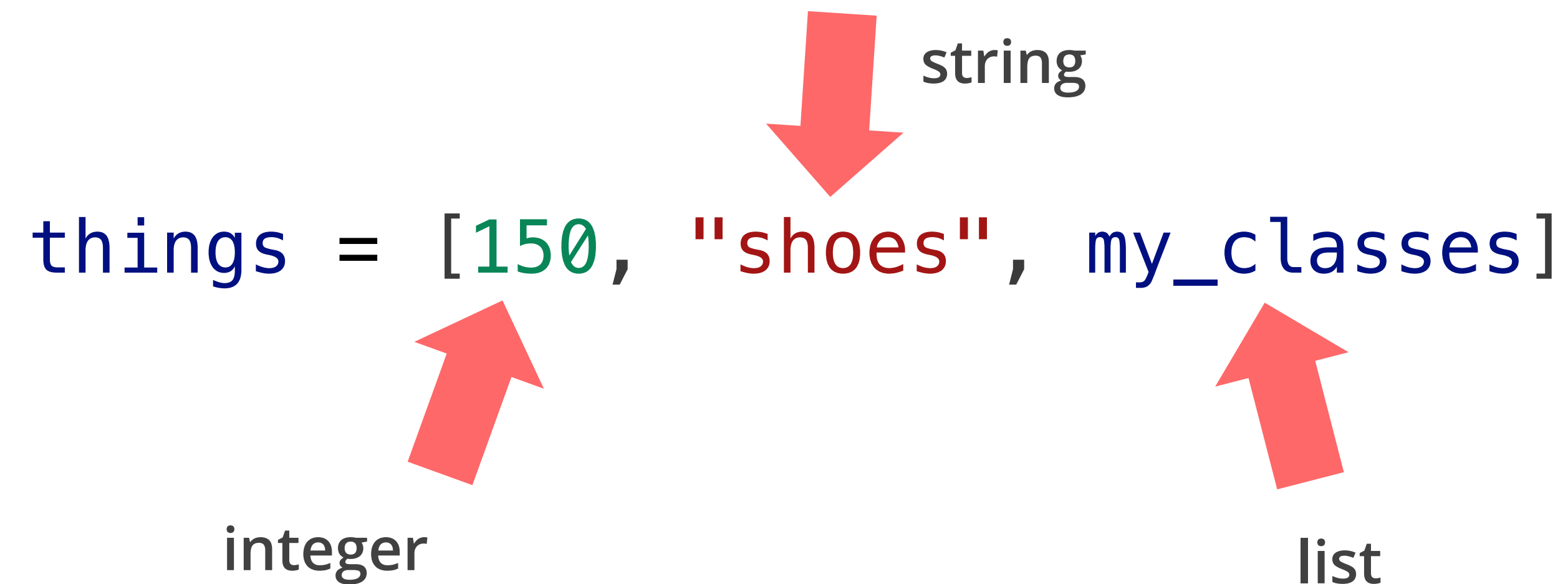
Python lists are a set of comma-separated values (items) between square brackets

```
my_classes = ["Computing for Data Analytics", "Database  
Management Systems", "Statistical Methods for Business"]
```

```
people = ["Mattan",  
          "Sarah",  
          "that person you forgot to text back",  
          "Samuel L. Jackson"]
```

What's in a list?

Lists can be any data type; each element doesn't need to be the same type



The diagram illustrates a Python list named `things` containing three elements: `150`, `"shoes"`, and `my_classes`. Red arrows point from each element to its respective data type label: `150` is labeled `integer`, `"shoes"` is labeled `string`, and `my_classes` is labeled `list`.

```
things = [150, "shoes", my_classes]
```

integer string list

Access items in a list

Lists can be indexed and sliced

```
things = [150, "shoes", my_classes]  
things[1] # what do you think this will return?
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]  
things[1] # returns shoes
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]
things[1] # returns shoes
things[0:2] # you can select a range of items
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]
things[1] # returns shoes
things[0:2] # you can select a range of items
things[2:] # or items starting at a certain position
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]
things[1] # returns shoes
things[0:2] # you can select a range of items
things[2:] # or items starting at a certain position
things[:2] # or up to a certain number of items
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]
things[1] # returns shoes
things[0:2] # you can select a range of items
things[2:] # or items starting at a certain position
things[:2] # or up to a certain number of items
things[-1] # even use negative values to get items from
the end of a list
```

Access items in a list

Lists can be indexed and sliced. The index of a list begins with 0 and ends with $n-1$, when n is the length of the list.

```
things = [150, "shoes", my_classes]
things[1] # returns shoes
things[0:2] # you can select a range of items
things[2:] # or items starting at a certain position
things[:2] # or up to a certain number of items
things[-1] # even use negative values to get items from
the end of a list
```

Convert strings to lists

```
# use .split() to turn a string into a list  
cities = "New York, San Francisco, London".split(", ")  
print(cities)
```

Convert strings to lists & vice versa

use `.split()` to turn a string into a list

```
cities = "New York, San Francisco, London".split(", ")  
print(cities)
```

use `.join()` to turn a list into a string

```
groceries = ["Milk", "Eggs", "Cheese"]  
print(", ".join(groceries))
```

Add to a list

Lists are a mutable type so it is possible to change their content.

```
more_things = [] # creates an empty list  
print(more_things) # returns brackets
```

Add to a list

Lists are a mutable type so it is possible to change their content. You can add new items at the end of the list, by using ***.append()***.

```
more_things = [] # creates an empty list
print(more_things) # returns brackets

more_things.append(1) # adds 1 to more_things
print(more_things) # returns 1 in brackets
```

Add to a list

Lists are a mutable type so it is possible to change their content. You can add new items at the end of the list, by using ***.append()***.

```
more_things = [] # creates an empty list  
print(more_things) # returns brackets
```

```
more_things.append(1) # adds 1 to more_things  
print(more_things) # returns 1 in brackets
```

```
more_things.append(11) # adds 11 to more_things  
print(more_things) # returns 1 and 11 in brackets
```

Other ways to add items

```
more_things.extend([10,12,13]) #adds several items to a list  
print(more_things)
```

Other ways to add items

```
more_things.extend([10,12,13]) #adds several items to a list  
print(more_things)
```

```
more_things.insert(3,15) #inserts an item at a specific location  
print(more_things)
```

Remove items from a list

```
more_things.remove(15) # removes the item 15
```

Remove items from a list

```
more_things.remove(15) # removes the item 15
```

```
more_things.pop(1) # removes and returns the item in the  
index position
```

```
more_things.pop() # removes and returns the last item in a  
list (default = -1)
```

Remove items from a list

```
more_things.remove(15) # removes the item 15
```

```
more_things.pop(1) # removes and returns the item in the  
index position
```

```
more_things.pop() # removes and returns the last item in a  
list (default = -1)
```

```
del more_things[2] # deletes the item in position 2
```

```
more_things.clear() # empties the list
```

```
del more_things # deletes the list variable
```

Remove items from a list

```
more_things.remove(15) # removes the item 15
```

```
more_things.pop(1) # removes and returns the item in the  
index position
```

```
more_things.pop() # removes and returns the last item in a  
list (default = -1)
```

```
del more_things[2] # deletes the item in position 2
```

```
more_things.clear() # empties the list
```

```
del more_things # deletes the list variable
```

Summarize list contents

```
more_things.count(15) # counts the number of times 15 occurs  
print("The count on 15 is ", more_things.count(15))
```

Summarize list contents

```
more_things.count(15) # counts the number of times 15 occurs  
print("The count on 15 is ", more_things.count(15))
```

```
len(more_things) # returns the number of items in a list
```

Summarize list contents

```
more_things.count(15) # counts the number of times 15 occurs  
print("The count on 15 is ", more_things.count(15))
```

```
len(more_things) # returns the number of items in a list
```

```
sum(more_things) # sums the items; only works for numbers
```

Summarize list contents

```
more_things.count(15) # counts the number of times 15 occurs  
print("The count on 15 is ", more_things.count(15))
```

```
len(more_things) # returns the number of items in a list
```

```
sum(more_things) # sums the items; only works for numbers
```

```
min(more_things) # returns the minimum value; for strings  
this base on alphabetical order
```

```
max(more_things) # returns the maximum value
```

Summarize list contents

```
more_things.count(15) # counts the number of times 15 occurs  
print("The count on 15 is ", more_things.count(15))
```

```
len(more_things) # returns the number of items in a list
```

```
sum(more_things) # sums the items; only works for numbers
```

```
min(more_things) # returns the minimum value; for strings  
this base on alphabetical order
```

```
max(more_things) # returns the maximum value
```

Change the order of lists

Note that these method change the original list

```
more_things.sort() # sort in ascending order  
print(more_things) # check out the result
```

Change the order of lists

Note that these method change the original list

```
more_things.sort() # sort in ascending order  
print(more_things) # check out the result
```

```
more_things.sort(reverse=True) # sort in descending order  
print(more_things)
```

Change the order of lists

Note that these methods change the original list

```
more_things.sort() # sort in ascending order  
print(more_things) # check out the result
```

```
more_things.sort(reverse=True) # sort in descending order  
print(more_things)
```

```
more_things.reverse() # reverse the items in a list  
print(more_things)
```

Change the order of lists

Note that these methods change the original list

```
more_things.sort() # sort in ascending order  
print(more_things) # check out the result
```

```
more_things.sort(reverse=True) # sort in descending order  
print(more_things)
```

```
more_things.reverse() # reverse the items in a list  
print(more_things)
```

```
import random # we've seen this package before  
random.shuffle(more_things)  
print(more_things)
```

Change the order of lists

Note that these methods change the original list

```
more_things.sort() # sort in ascending order  
print(more_things) # check out the result
```

```
more_things.sort(reverse=True) # sort in descending order  
print(more_things)
```

```
more_things.reverse() # reverse the items in a list  
print(more_things)
```

```
import random # we've seen this package before  
random.shuffle(more_things)  
print(more_things)
```

Order but retain original list

Consider these list functions:

```
sorted(more_things)
```

```
list(reversed(more_things))
```

Questions on these

Accessing items in a list

Converting strings to lists & vice versa

Adding items to a list

Removing items from a list

Summarizing a list

Sorting a list

**How does this all
apply or relate to
data analysis?**

Challenge 1: shuffle

Starting with the string “MSFT, TSLA, GOOG, AAPL” do the following:

1. *Split* it into a list
2. Randomly *shuffle* the list
3. *Join* it back together into a string
4. Make the string *lowercase*
5. *Print* out the result

Let's discuss

Starting with the string “MSFT, TSLA, GOOG, AAPL” do the following:

1. *Split* it into a list
2. Randomly *shuffle* the list
3. *Join* it back together into a string
4. Make the string *lowercase*
5. *Print* out the result

One possible solution

```
import random
my_string = "MSFT, TSLA, GOOG, AAPL"

# 1. Split it into a list
my_list = my_string.split(", ")

# 2. Randomly shuffle the list
random.shuffle(my_list)

# 3. Join it back together into a string
new_string = ", ".join(my_list)

# 4. Make the list lowercase
new_string = new_string.lower()

# 5. Print out the result
print(new_string)
```

This week

This week we will explore Python **lists** and learn about how to apply **looping logic** in Python

for loops

Python's ***for*** statement iterates over the items of any sequence (a list or a string), in the order that they appear.

```
words = ['cat', 'window', 'defenestrate']  
for w in words:  Note colon  
& indent print(w, len(w))
```

output:

```
cat 3  
window 6  
defenestrate 12
```

for loops

Python's ***for*** statement iterates over the items of any sequence (a list or a string), in the order that they appear.

```
words = ['cat', 'window', 'defenestrate']  
for w in words:  need a looping variable & a list/string  
    print(w, len(w))
```

output:

```
cat 3  
window 6  
defenestrate 12
```

for loop facts

- A *for* loop is made up of a collection (e.g. list, string), a loop variable, and a body.
- The first line of the *for* loop must end with a colon, and the body must be indented—indentation is always meaningful in Python.
- A *for* loop executes commands once for each value in a list, a character string, or some other collection.
- Loop variables can be called anything but it is strongly advised to have a meaningful name to the looping variable.
- The body of a loop can contain many statements.
- Use `range` to iterate over a sequence of numbers.

Range() function

If you do need to iterate over a sequence of numbers, the built-in function ***range()*** comes in handy

```
for i in range(5):  
    print(i)
```

output:

```
0  
1  
2  
3  
4
```

Note that range begins with 0

Specify range and increments

It is possible to let the *range()* start at specific number, or to step through different increments (even negative)

```
list(range(5, 10)) #[5, 6, 7, 8, 9]
```

Specify range and increments

It is possible to let the *range()* start at specific number, or to step through different increments (even negative)

```
list(range(5, 10)) #[5, 6, 7, 8, 9]
```

```
list(range(0, 10, 3)) #[0, 3, 6, 9]
```

Specify range and increments

It is possible to let the *range()* start at specific number, or to step through different increments (even negative)

```
list(range(5, 10)) #[5, 6, 7, 8, 9]
```

```
list(range(0, 10, 3)) #[0, 3, 6, 9]
```

```
list(range(-10, -100, -30)) #[-10, -40, -70]
```

Loop control statements

- break
- continue
- pass

Break a loop

A **break** statement is use to immediately terminate a loop

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

Continue a loop

A **continue** statement is use to skip out future commands inside a loop and return to the top of the loop

```
for i in range(5):  
    if i < 3:  
        continue  
    print(i)
```

Pass statement

A **pass** is a null operation where nothing happens. It can be a useful placeholder when a statement is required syntactically when no code needs to be executed.

```
for i in range(5):  
    pass
```

Combine for loops and if statements

```
even = []  
  
for i in range(5):  
    if i%2 == 0: # % is the modulo operator  
        even.append(i)  
  
print("The even numbers are ", even)
```

Note indentations here!

while loops

With the **while** loop we can execute a set of statements as long as a condition is true.

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Replace a value in a list

```
# replace banana with orange  
a_list = ['apple', 'banana', 'cherry']
```

Replace a value in a list

```
# replace banana with orange
a_list = ['apple', 'banana', 'cherry']

i = 0
while i < len(my_list):
    if a_list[i] == 'banana':
        a_list[i] = 'orange'
    i += 1
```

Note indentations here!

Which loop to use?

<i>For</i>	<i>While</i>
Use to iterate over a sequence of items .	Use to repeatedly execute a block of statements while a condition is true .

Which loop to use?

<i>For</i>	<i>While</i>
Use to iterate over a sequence of items .	Use to repeatedly execute a block of statements while a condition is true .
Designed for iterating over a sequence of items. Eg. list, etc.	Useful when the number of iterations is not known or when we want to repeat a block of code until a certain condition is met .

Questions?

Challenge 2: print squares

Write code to print out the squares of the numbers 4 to 15

Let's discuss

Write code to print out the squares of the numbers 4 to 15

One possible solutions

```
# this code prints out the squares of the numbers 4 to 15
```

```
for number in range(4, 16): # range to isolate the values  
    print(number, "squared is", number * number)
```

Next week

bespoke functions in Python